

Name: Raul Dmonty

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import seaborn as sns
titanic_data=sns.load_dataset('titanic')
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class \
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third
..	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	13.0000	S	Second
887	1	1	female	19.0	0	0	30.0000	S	First
888	0	3	female	NaN	1	2	23.4500	S	Third
889	1	1	male	26.0	0	0	30.0000	C	First
890	0	3	male	32.0	0	0	7.7500	Q	Third

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True
..	...	...	...	...	...	...
886	man	True	NaN	Southampton	no	True
887	woman	False	B	Southampton	yes	True
888	woman	False	NaN	Southampton	no	False
889	man	True	C	Cherbourg	yes	True
890	man	True	NaN	Queenstown	no	True

```
titanic_data.shape
```

```
titanic_data.columns
```

```
titanic_data.head()
```

```
titanic_data.tail()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive
886	0	2	male	27.0	0	0	13.00	S	Second	man	True	NaN	Southampton	no
887	1	1	female	19.0	0	0	30.00	S	First	woman	False	B	Southampton	yes
888	0	3	female	NaN	1	2	23.45	S	Third	woman	False	NaN	Southampton	no
889	1	1	male	26.0	0	0	30.00	C	First	man	True	C	Cherbourg	yes
890	0	3	male	32.0	0	0	7.75	Q	Third	man	True	NaN	Queenstown	no

```
titanic_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    object
8   class       891 non-null    category
9   who         891 non-null    object
10  adult_male  891 non-null    bool
11  deck        203 non-null    category
12  embark_town 889 non-null    object
13  alive       891 non-null    object
14  alone       891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

```
missing_values=titanic_data.isnull().sum()
print(missing_values)
```

```
survived      0
pclass        0
sex           0
age          177
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town   2
alive         0
alone         0
dtype: int64
```

```
data = titanic_data['fare']
mean = np.mean(data) # calculates the mean
std_dev = np.std(data) # calculates the standard deviation
Z_scores = (data - mean) / std_dev # computes the Z-scores
outliers = data[np.abs(Z_scores) > 3]
print(outliers)
```

```
27    263.0000
88    263.0000
118   247.5208
258   512.3292
299   247.5208
311   262.3750
341   263.0000
377   211.5000
380   227.5250
438   263.0000
527   221.7792
557   227.5250
679   512.3292
689   211.3375
700   227.5250
716   227.5250
730   211.3375
737   512.3292
742   262.3750
779   211.3375
Name: fare, dtype: float64
```

```
# Normalize 'age'
titanic_data['age'] = (titanic_data['age'] - titanic_data['age'].min()) / (titanic_data['age'].max() - titanic_data['age'].min())

# Display the normalized ages
print(titanic_data['age'])
```

```
0      0.271174
1      0.472229
2      0.321438
3      0.434531
4      0.434531
...
886    0.334004
887    0.233476
888         NaN
889    0.321438
890    0.396833
Name: age, Length: 891, dtype: float64
```

```
# Standardize 'fare'
titanic_data['fare'] = (titanic_data['fare'] - titanic_data['fare'].mean()) / titanic_data['fare'].std()

# Display the standardized fares
print(titanic_data['fare'])
```

```
0      -0.502163
1       0.786404
2      -0.488580
3       0.420494
4      -0.486064
...
886    -0.386454
887    -0.044356
888    -0.176164
889    -0.044356
890    -0.492101
Name: fare, Length: 891, dtype: float64
```

```
new_titanic_df=titanic_data.drop(columns=['deck'])
```

```
new_titanic_df['age'].fillna(new_titanic_df['age'].median(),inplace=True)
```

C:\Users\HP\AppData\Local\Temp\ipykernel_13280\1870140451.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series consisting of rows that may not be sorted.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which it is applied is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col].method(value, inplace=True)'.

```
new_titanic_df['age'].fillna(new_titanic_df['age'].median(),inplace=True)
```

```
missing_values=new_titanic_df.isnull().sum()
print(missing_values)
```

```
survived      0
pclass        0
sex            0
age           0
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
embark_town   2
alive         0
alone         0
dtype: int64
```

```
data=new_titanic_df
data['embark_town'].dtype
data['embark_town'].unique()
data['embark_town'].fillna(data['embark_town'].mode()[0],inplace=True)
data.isnull().sum()
```

C:\Users\HP\AppData\Local\Temp\ipykernel_13280\1765232976.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series consisting of rows that may not be sorted.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which it is applied is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col].method(value, inplace=True)'.

```
data['embark_town'].fillna(data['embark_town'].mode()[0],inplace=True)
```

```

survived      0
pclass        0
sex            0
age           0
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
embark_town   0
alive         0
alone         0
dtype: int64

```

```

data=new_titanic_df
data['embarked'].dtype
data['embarked'].unique()
data['embarked'].fillna(data['embarked'].mode()[0],inplace=True)
data.isnull().sum()

```

C:\Users\HP\AppData\Local\Temp\ipykernel_13280\1721811771.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series, and its current dtype is not a float. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which it would operate would be a view.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or 'df[col] = df[col].method(value)'

```

data['embarked'].fillna(data['embarked'].mode()[0],inplace=True)
survived      0
pclass        0
sex            0
age           0
sibsp         0
parch         0
fare          0
embarked      0
class         0
who           0
adult_male    0
embark_town   0
alive         0
alone         0
dtype: int64

```

```

le=LabelEncoder()
data['sex']=le.fit_transform(data['sex'])
data['embarked']=le.fit_transform(data['embarked'])

```

```
data.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone
0	0	3	1	22.0	1	0	-0.502163	2	Third	man	True	Southampton	no	False
1	1	1	0	38.0	1	0	0.786404	0	First	woman	False	Cherbourg	yes	False
2	1	3	0	26.0	0	0	-0.488580	2	Third	woman	False	Southampton	yes	True
3	1	1	0	35.0	1	0	0.420494	2	First	woman	False	Southampton	yes	False
4	0	3	1	35.0	0	0	-0.486064	2	Third	man	True	Southampton	no	True

```
data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  -
0   survived        891 non-null   int64
1   pclass          891 non-null   int64
2   sex             891 non-null   int64
3   age            891 non-null   float64
4   sibsp          891 non-null   int64
5   parch          891 non-null   int64
6   fare           891 non-null   float64
7   embarked        891 non-null   int64
8   class           891 non-null   category
9   who             891 non-null   object
10  adult_male      891 non-null   bool
11  embark_town     891 non-null   object
12  alive           891 non-null   object
13  alone           891 non-null   bool

```

```
dtypes: bool(2), category(1), float64(2), int64(6), object(3)
memory usage: 79.4+ KB
```

```
data=data[['pclass','sex','age','fare','embarked','survived']]
X=data[['pclass','sex','age','fare','embarked']]
y=data['survived']
```

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.3, random_state=42)
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=1000)
```

```
y_pred=model.predict(X_test)
```

```
accuracy=accuracy_score(y_test, y_pred)
print("Accuracy:",accuracy)
```

```
Accuracy: 0.7947761194029851
```

```
new_passenger=pd.DataFrame({
    'pclass':[3],
    'sex':['male'],
    'age':[28],
    'fare':[7.25],
    'embarked':['S']
})
```

```
new_passenger_encoded=pd.get_dummies(new_passenger)
new_passenger_encoded=new_passenger_encoded.reindex(columns=X.columns, fill_value=0)
```

```
prediction=model.predict(new_passenger_encoded)
print("Survived" if prediction[0]==1 else "Did Not Survive")
```

```
Survived
```

```
new_passengers=pd.DataFrame({
    'pclass':[1,2,3],
    'sex':['female','male','female'],
    'age':[38,45,14],
    'fare':[80.0,8.05,20.0],
    'embarked':['C','S','Q']
})
```

```
new_passengers_encoded=pd.get_dummies(new_passengers)
new_passengers_encoded=new_passengers_encoded.reindex(columns=X.columns, fill_value=0)
```

```
predictions=model.predict(new_passengers_encoded)
for i,pred in enumerate(predictions):
    print(f"Passenger {i+1}:",
          "Survived" if pred ==1 else "Did not Survive")
```

```
Passenger 1: Survived
Passenger 2: Survived
Passenger 3: Survived
```

Start coding or [generate](#) with AI.